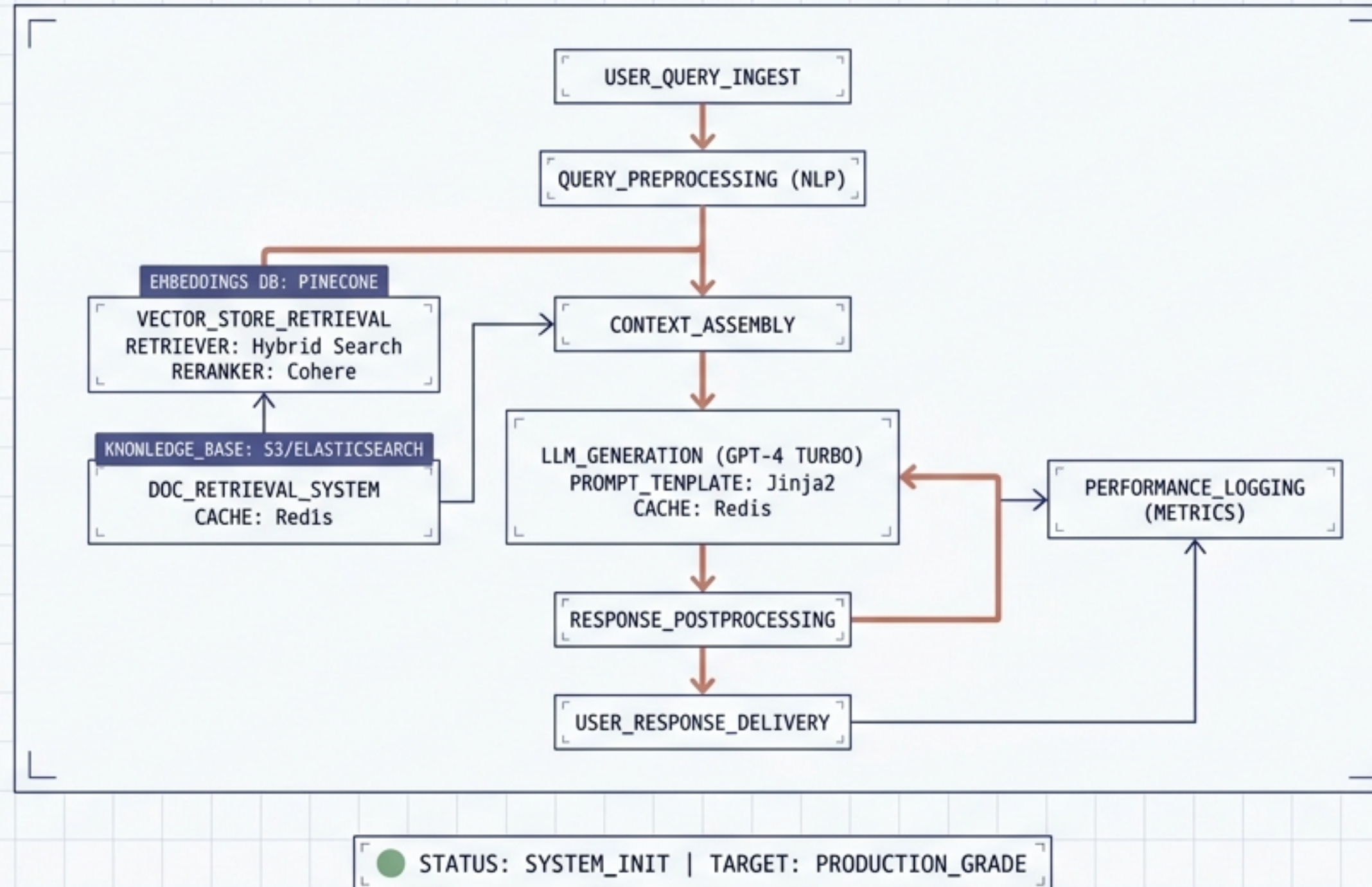
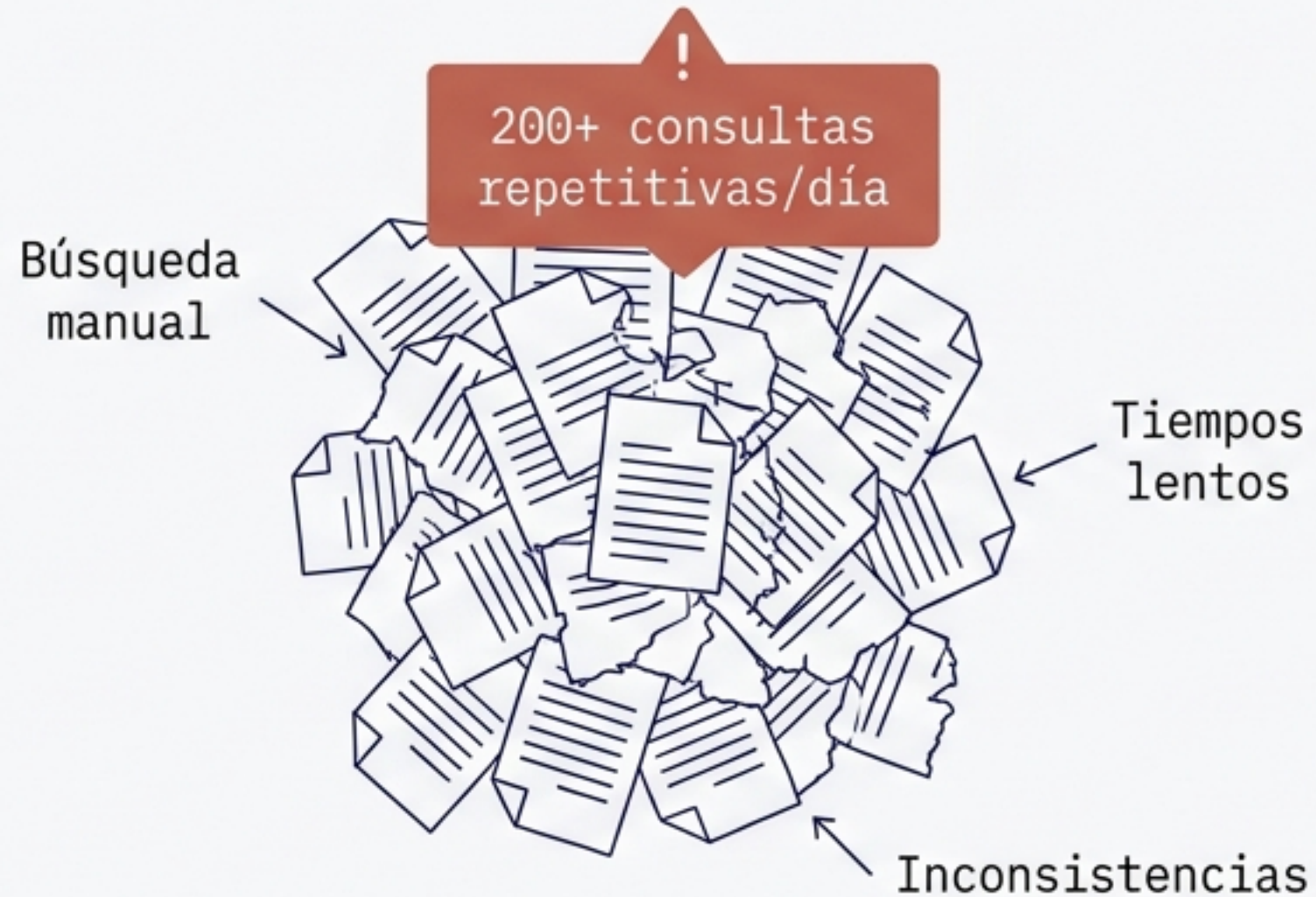


Blueprint de Arquitectura RAG

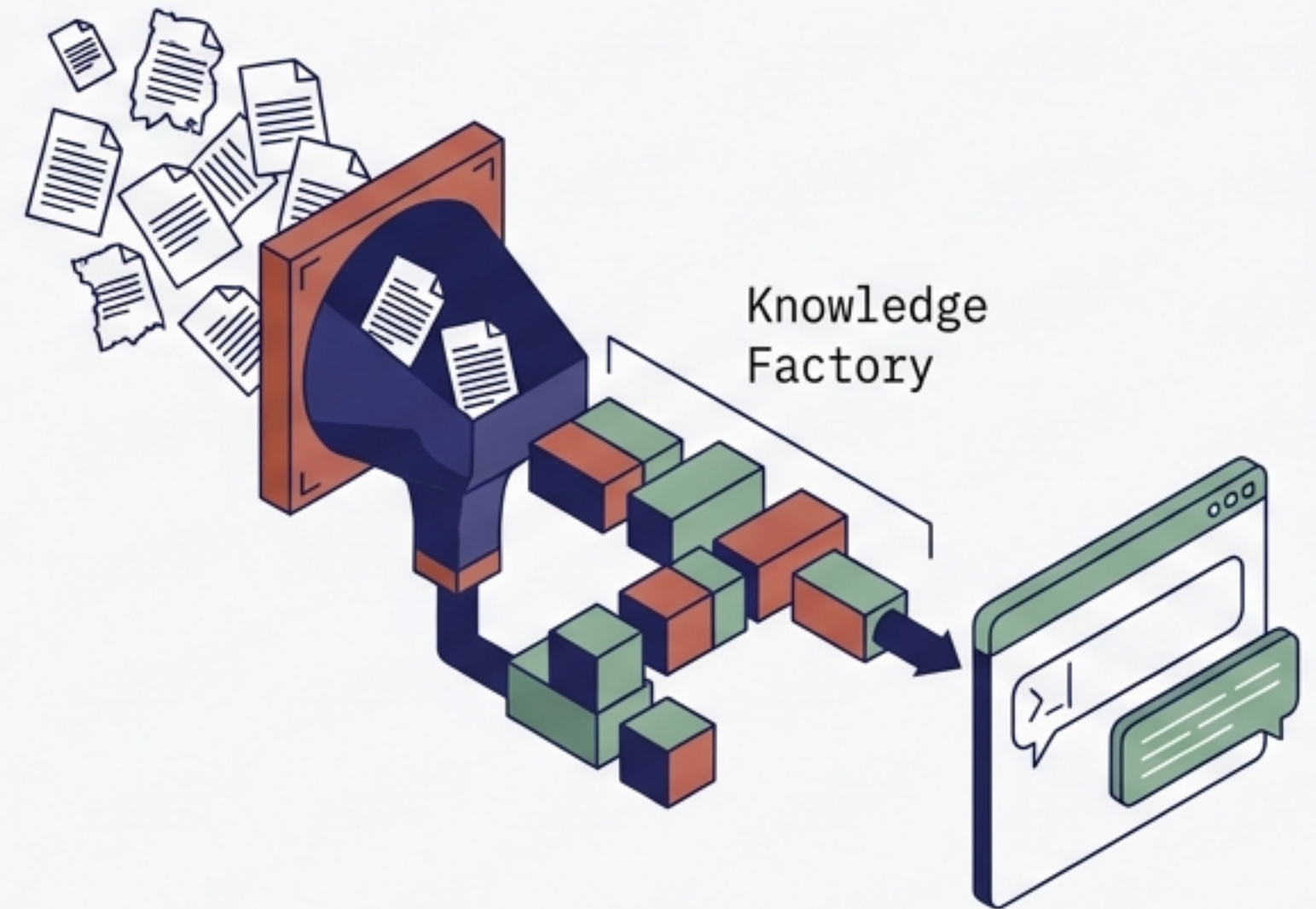
Especificaciones técnicas y contrato de sistema para el Chatbot de Soporte FAQ.



De la entropía documental a la inteligencia estructurada

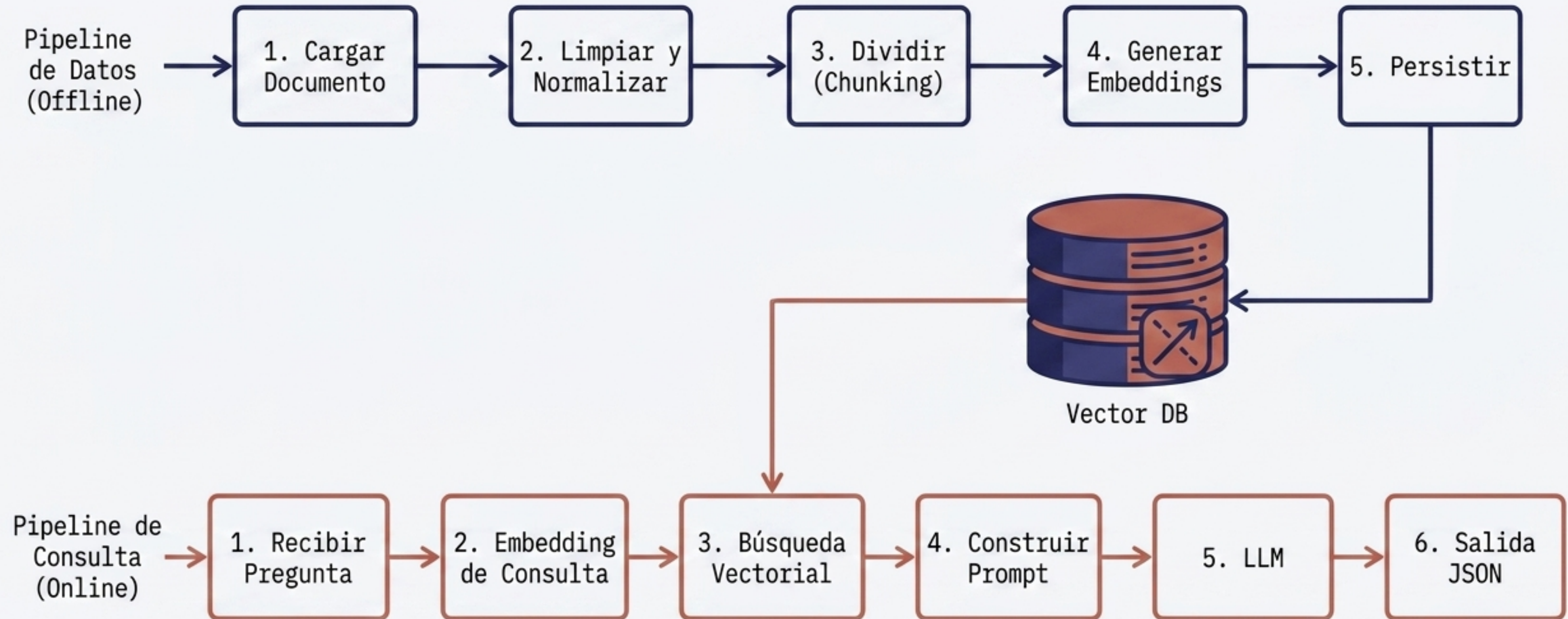


El problema actual: Dependencia del conocimiento individual y alto costo operativo.

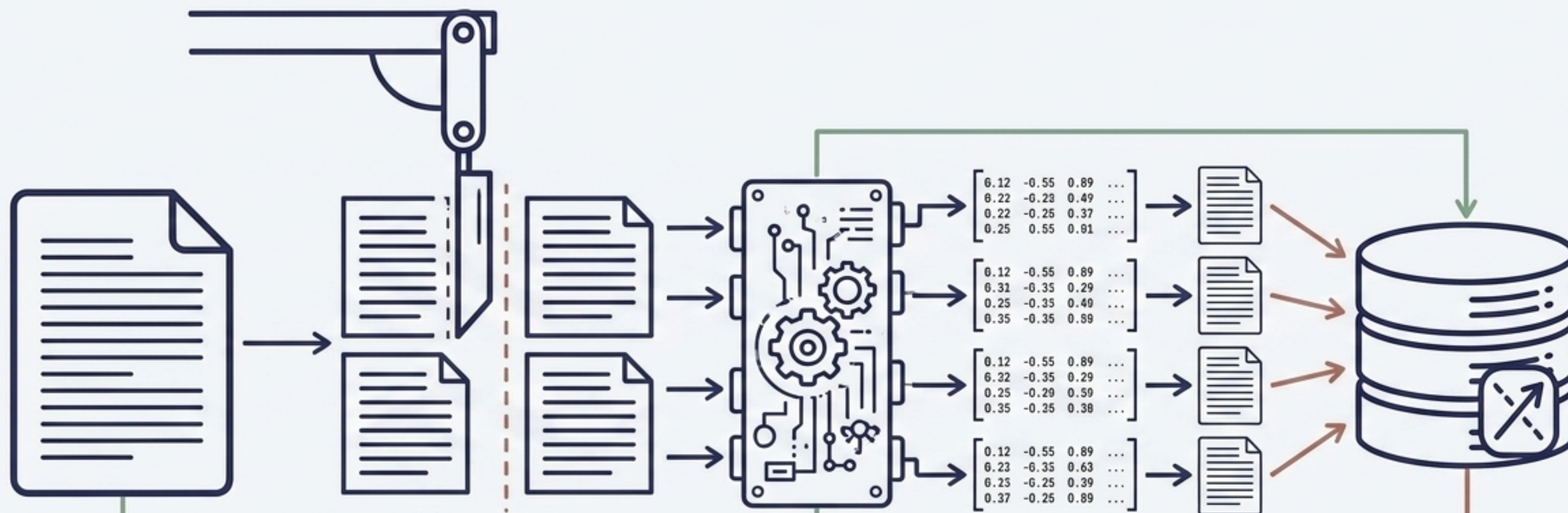
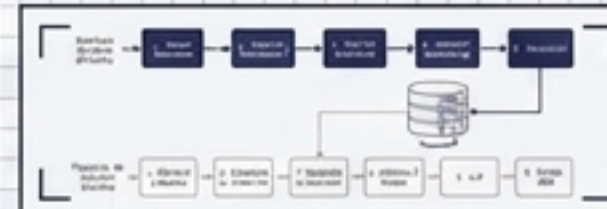


La solución RAG: Transformación de texto plano en una base de conocimiento vectorial consultable y automatizada.

Arquitectura de Alto Nivel: El flujo de transformación



Pipeline de Indexación: Preparando la materia prima



Fuente: Documento de texto plano (políticas, procedimientos, FAQs). Mínimo 1.000 palabras.

Segmentación:
Generar ≥ 20 chunks significativos.

Vectorización:
Validación estricta: N chunks = N embeddings.

Almacenamiento:
Persistir texto original, metadata y vectores unificados.

Matriz de Decisión: Estrategias de Segmentación (Chunking)

Estrategia	Funcionamiento	Complejidad / Desventajas	Veredicto
Tamaño fijo con overlap	Bloques de 50-500 tokens con superposición	Fácil de reproducir. Conserva continuidad	RECOMENDADO Recomendado para el proyecto.
Por párrafos	División por saltos de línea natural	Inconsistencia de tamaño. Documento dependiente	Útil si el formato es muy estricto.
Por oraciones	División por signos de puntuación	Demasiado fragmentado. Pierde contexto global	No recomendado como base.
Semántico	Agrupación por similitud de embeddings	Alta complejidad. Requiere pre-procesamiento	Excesivo para este nivel.

Anatomía de un Chunk: El poder del Overlap

Chunk 1

La fragmentación de documentos extensos en unidades más pequeñas, conocidas como "chunks", es un paso fundamental en los sistemas de recuperación de información y generación aumentada. Esta técnica permite que los modelos de lenguaje procesen información relevante de manera eficiente, mejorando la precisión de las respuestas. Sin embargo, un desafío crítico es la pérdida de contexto en los puntos de corte entre chunks adyacentes. Para mitigar esto, se implementa la superposición o "overlap", una estrategia que asegura que una porción de texto se repita al final de un chunk y al principio del siguiente. Esta redundancia calculada preserva la continuidad semántica y las relaciones entre frases, evitando que información crucial quede aislada o malinterpretada.

El tamaño del overlap es un parámetro ajustable que impacta directamente en el rendimiento del sistema, equilibrando la integridad contextual con la eficiencia del procesamiento y el uso de memoria. Una configuración óptima de overlap garantiza que el modelo siempre tenga acceso al contexto necesario, independientemente de dónde caiga el límite del chunk, lo que resulta en un sistema más robusto y capaz de manejar consultas complejas que requieren comprensión de múltiples segmentos de información.

El *overlap* preserva el contexto semántico que de otro modo se perdería en el punto de corte.

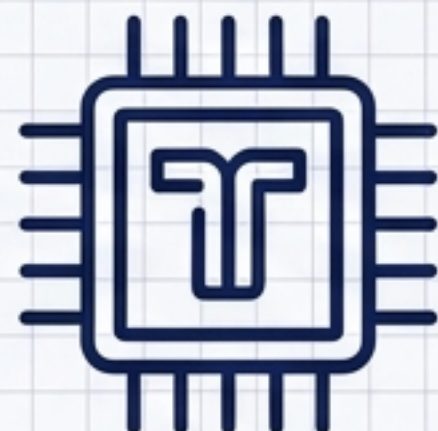
Chunk 2

Tamaño objetivo: 50 a 500 tokens por chunk.

Procesamiento: 100% del documento debe estar contenido en el índice final.

Generación de Vectores y Persistencia

Modelos Permitidos



API de OpenAI
Sentence-Transformers
o modelo local documentado.

Arquitectura de Almacenamiento

NumPy

FAISS



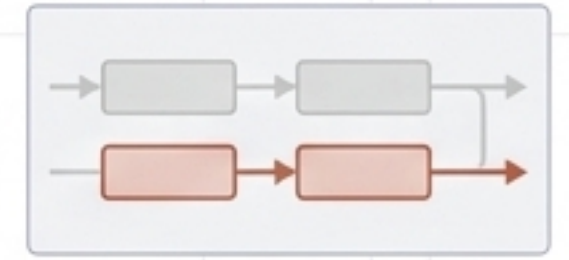
Chroma

Qdrant

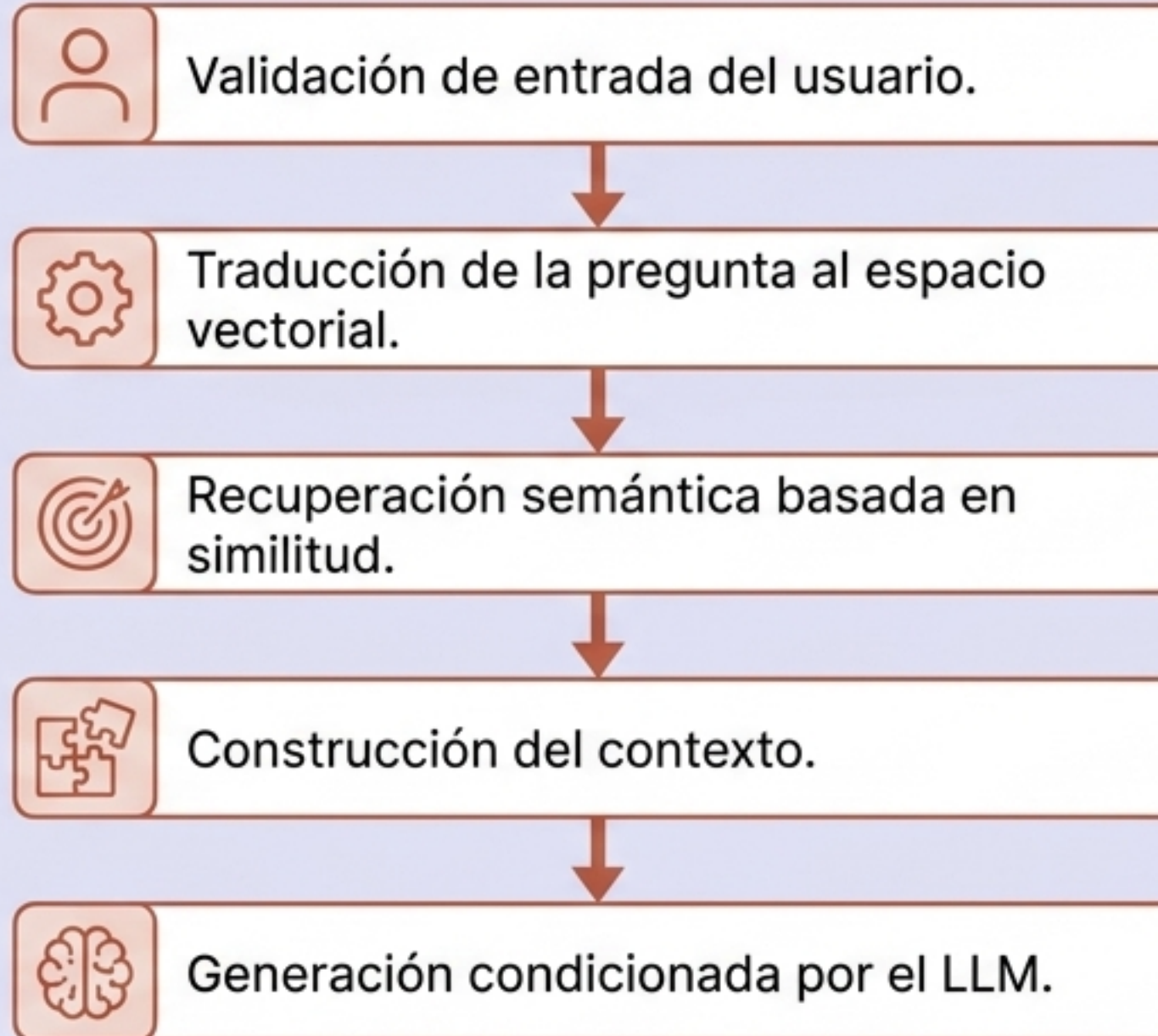


Regla Inquebrantable: La consulta del usuario DEBE utilizar exactamente el mismo modelo de embeddings que se utilizó para indexar los documentos. De lo contrario, la geometría colapsa.

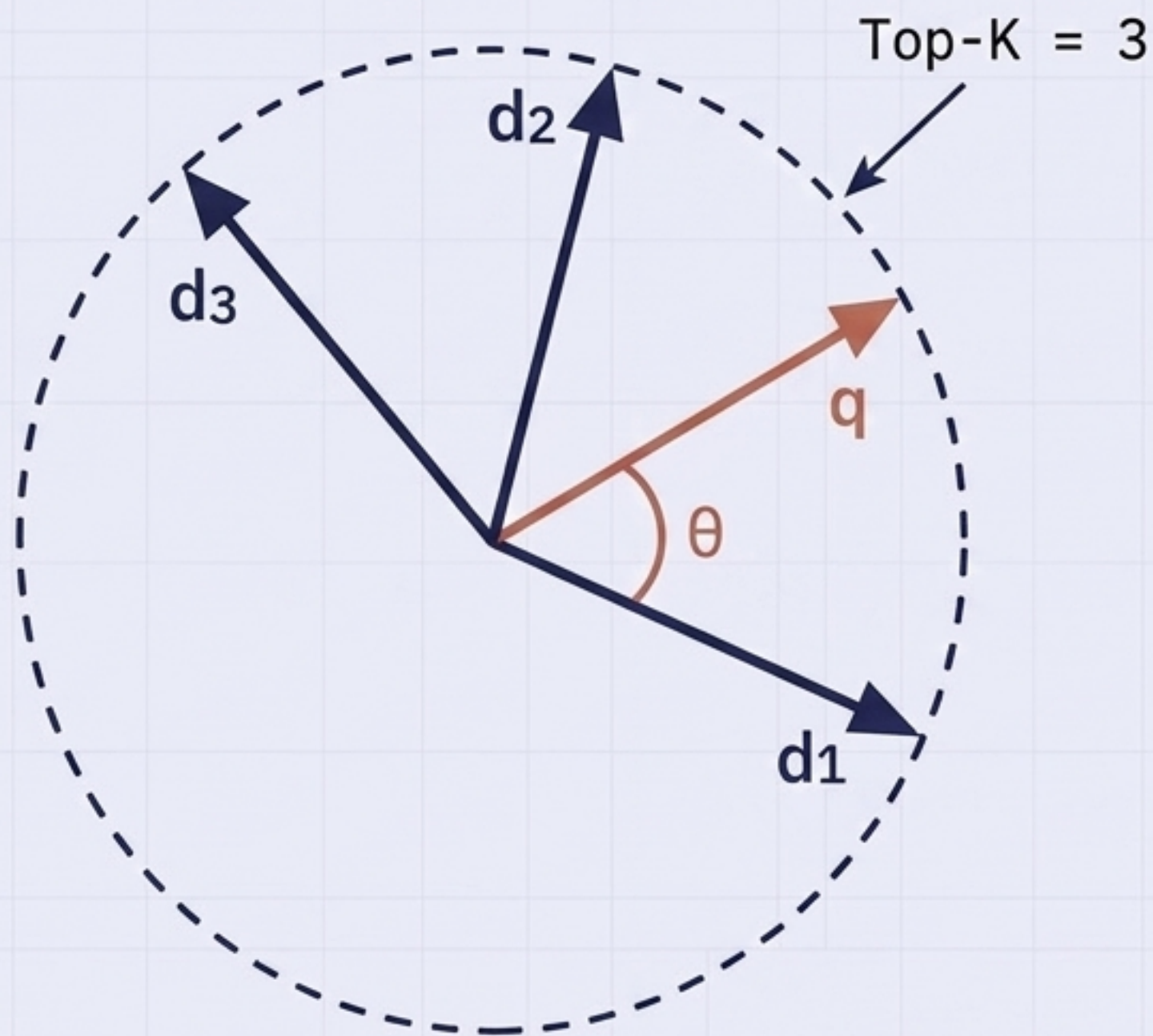
Pipeline de Consulta: Ensamblaje en tiempo real



Proceso Crítico (Online)



Búsqueda Vectorial: Similitud Coseno y Filtro Top-K

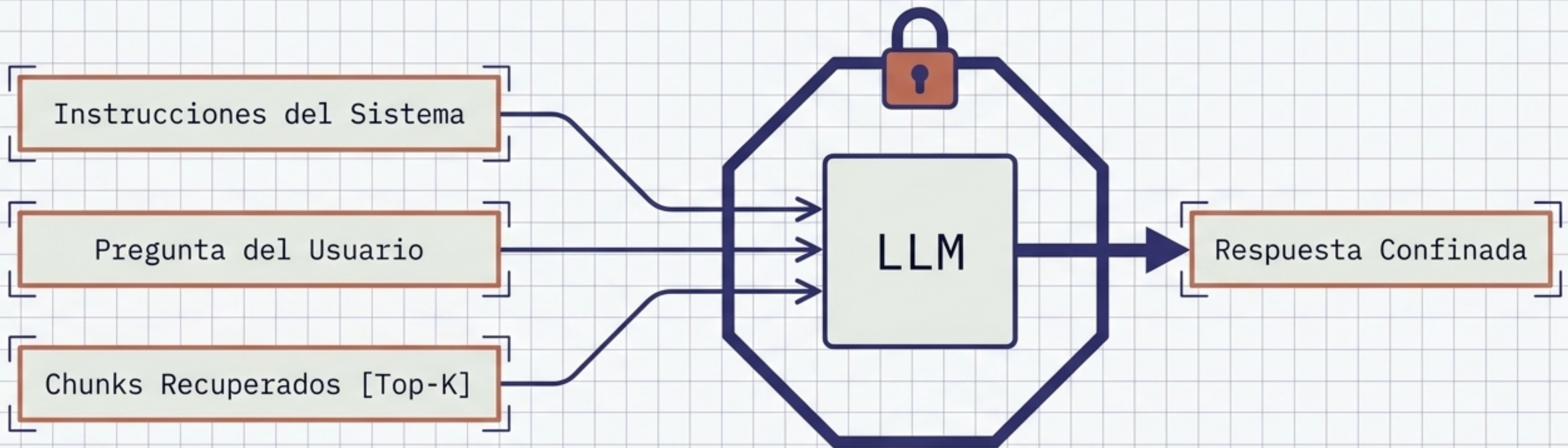


$$\text{cosine}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

Parámetros de Relevancia

- ⚙️ **Top-K:** El sistema debe recuperar exactamente entre 2 y 5 chunks por consulta.
- **Umbral de Calidad:** Al menos 80% de los chunks recuperados deben ser directamente relevantes a la pregunta.
- **Opción mínima obligatoria:** k-NN con similitud coseno.

Construcción del Prompt: El cerco lógico (Grounding)



Instrucción Core: El *prompt* debe obligar al modelo a responder **exclusivamente** usando el **contexto provisto**.

Manejo de vacíos (Threshold): Si no hay **resultados relevantes** o el **contexto es insuficiente**, el modelo debe indicar que **no tiene información**, **nunca inventarla**.

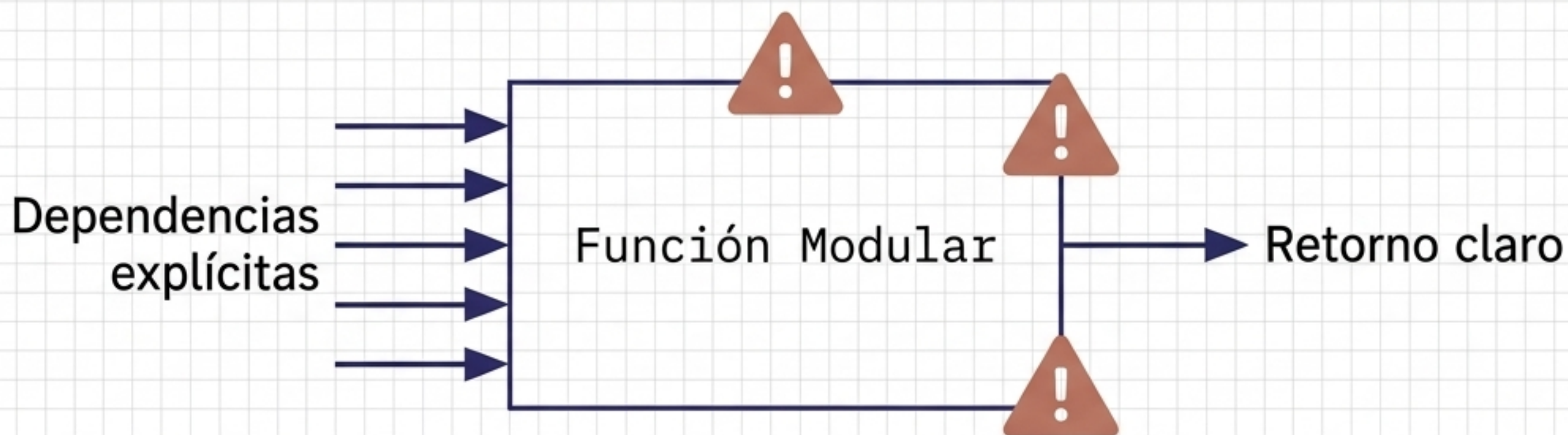
El Contrato JSON: Estructura de salida inquebrantable

```
{  
  "user_question": "¿Cómo recupero mi contraseña?",  
  "system_answer": "Para recuperar tu contraseña, debes ir a...",  
  "chunks_related": ["chunk_id_4", "chunk_id_12"]  
}
```

**CONTRACT
VALID**

Regla Estricta: No deben agregarse claves adicionales en la respuesta principal. Cualquier campo adicional rompe el contrato del proyecto.

[Modularidad y Resiliencia del Sistema]



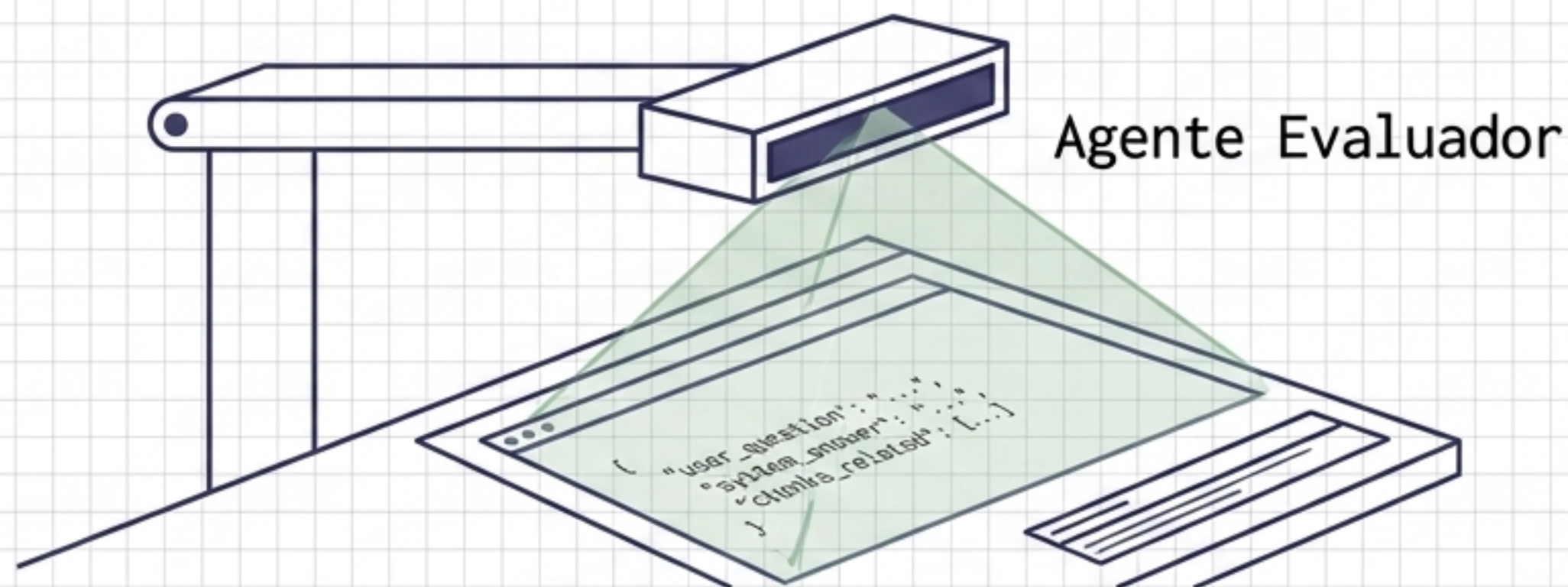
Reglas de Diseño (Mínimo 4 funciones):

- Una única responsabilidad por función (< 30 líneas idealmente).
- Evitar modificar el estado global.

Manejo de Errores Requerido”:

- Archivo fuente inexistente / vacío.
- API key faltante o error del proveedor.
- Índice no construido o JSON inválido generado por el LLM.

Agente Evaluador (Bonus): Control de Calidad Autónomo

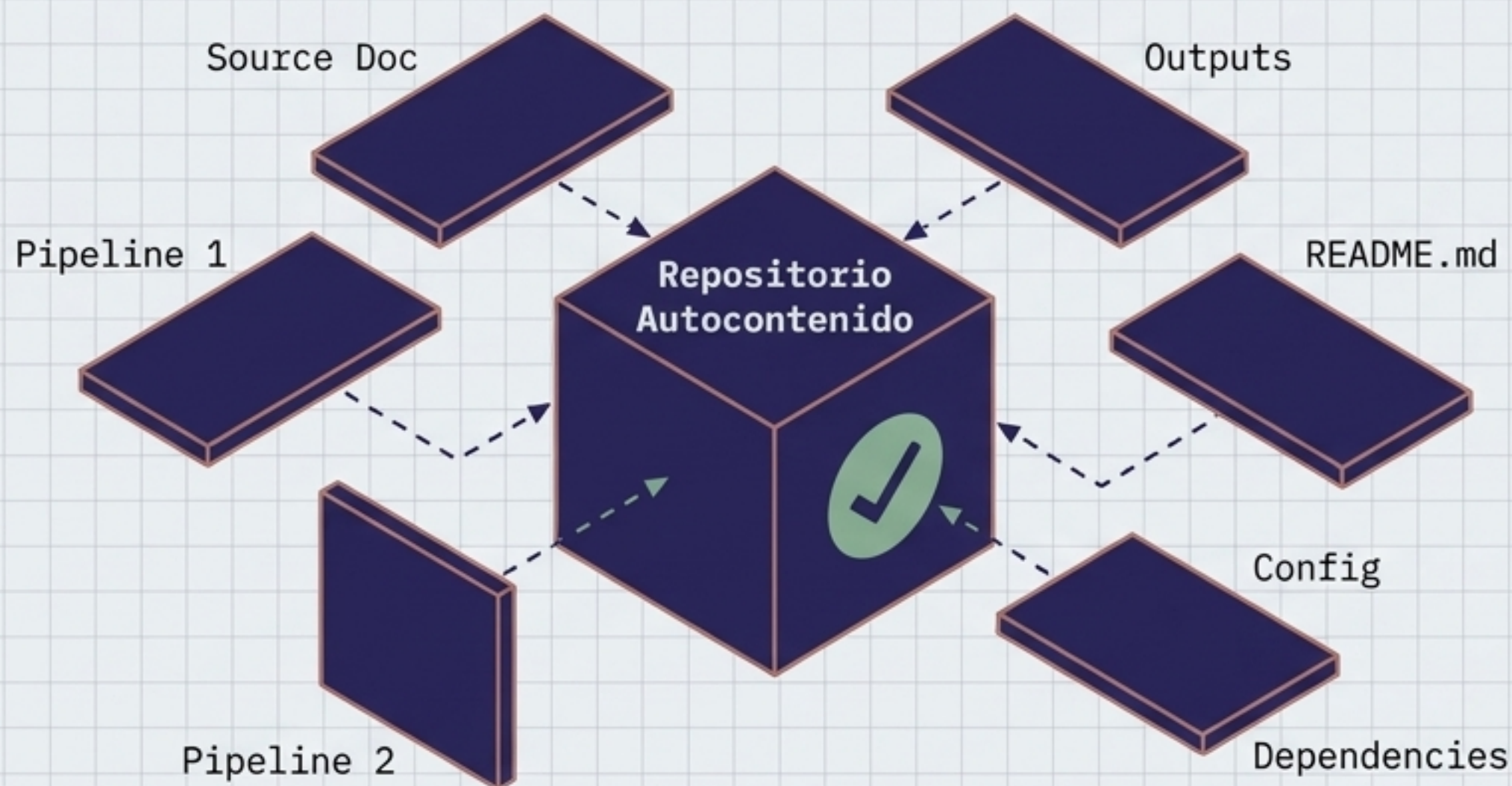


REQUISITOS TÉCNICOS

- Input: Pregunta + Respuesta + Chunks relacionados.
- Dimensiones Analizadas (mínimo 2): Relevancia, Precisión, Grounding, Completitud.
- Contrato de Evaluación:
 - score: Entero entre 0 y 10.
 - reason: Justificación concreta (mínimo 50 caracteres).

[El estándar de Reproducibilidad]

“Un sistema que funciona en tu máquina, pero no puede ser clonado y ejecutado por otro, no es un sistema completo.”



Contenido Crítico del Repositorio:

- requirements.txt con versiones exactas.
- .env.example (sin secretos reales).
- README.md con instrucciones de instalación y decisiones técnicas justificadas.

[Definition of Done: Criterios de Aceptación]



Calidad de Datos

>1000 palabras,
>20 chunks
(50-500 tokens),
Embeddings
unificados.



Lógica de Recuperación

Método
implementado
(e.g., Cosine),
Top-K 2-5, 80%
relevancia.



Fidelidad de Salida

Contexto
ensamblado,
prompt con
grounding, JSON
estricto (3
claves exactas).



Ingeniería y Entrega

Código modular,
manejo de
errores, README
completo, entorno
replicable
limpiamente.